

References as class members

- Declared without initial value

References as class members

- Declared without initial value
- Must be initialized using constructor initializer list

```
class X {  
public:  
    int& m_y;  
    X(int& a) ;  
};  
X::X(int& a) : m_y(a) { }
```

code & demo

Returning references

- Functions can return references
 - But they should refer to non-local variables!

```
#include <cassert>
const int SIZE = 32;
double myarray[SIZE];
double& subscript(const int i) {
    return myarray[i];
}
```

Example

```
main() {  
    for (int i = 0; i < SIZE; i++) {  
        myarray[i] = i * 0.5;  
    }  
    double value = subscript(12);  
    subscript(3) = 34.5;  
}
```

const in functions arguments

- Passing by const value – don't do it
- Passing by const reference

Person(const string& name, int weight);

- won't change the string object
- more efficient than pass by value (copy)
- const qualifier protects from change

const reference parameters

- What if you don't want to change the argument
- Use ***const*** modifier

```
// y is a constant! Can't be modified
void func(const int& y, int& z) {
    z = z * 5; // ok
    y += 8; // error!
};
```

Temporary values are const

- What you type

```
void func(int &);  
func(i * 3); // Generates warning or error!
```

- What the compiler generates

```
void func(int &);  
const int _tmp_ = i * 3;  
func(_tmp_); // Problem - binding non-const ref  
             // to const argument!
```

The temporary is constant, since you can't access it

const in function returns

- return by const value
 - basically it means nothing
- return by const pointer or reference
 - depends on what you want your client to do with the return value

code & demo

Copy Ctor

Object-Oriented Programming with C++

Copying

- Create a new object from an existing one
 - For example, when calling a function

```
// Currency as pass-by-value argument
```

```
void func(Currency p) {
```

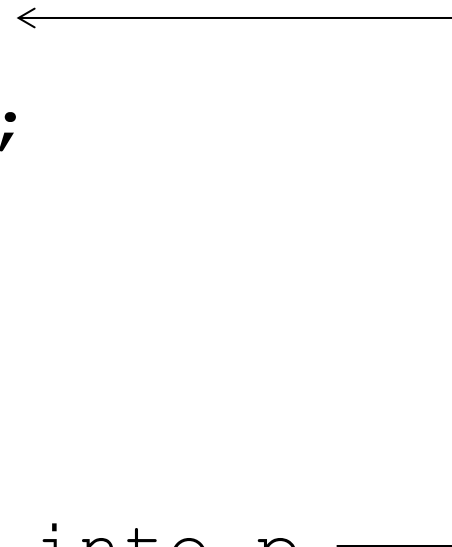
```
    cout << "X = " << p.dollars();
```

```
}
```

```
...
```

```
Currency bucks(100, 0);
```

```
func(bucks); // bucks is copied into p
```



Example: HowMany.cpp

The copy constructor

The copy constructor

- Copying is implemented by the *copy constructor*

The copy constructor

- Copying is implemented by the *copy constructor*
- Has the unique signature

```
T::T(const T&) ;
```

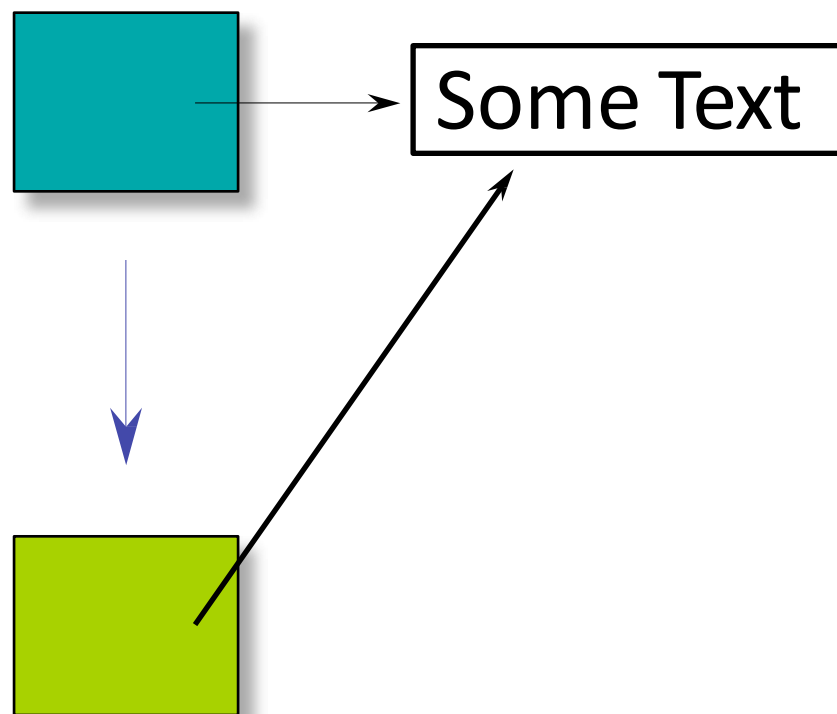
 - Call-by-reference is used for the explicit argument
- C++ builds a copy ctor for you if you don't provide one!
 - Copies each member variable
 - Good for numbers, objects, object arrays
 - Copies each pointer
 - Data may become shared!

What if class contains pointers?

```
class Person {  
public:  
    Person(const char *s);  
    ~Person();  
    void print();  
    // ... accessor functions  
private:  
    char *name;    // char * instead of string  
    //... more info e.g. age, address, phone  
};
```

Choices

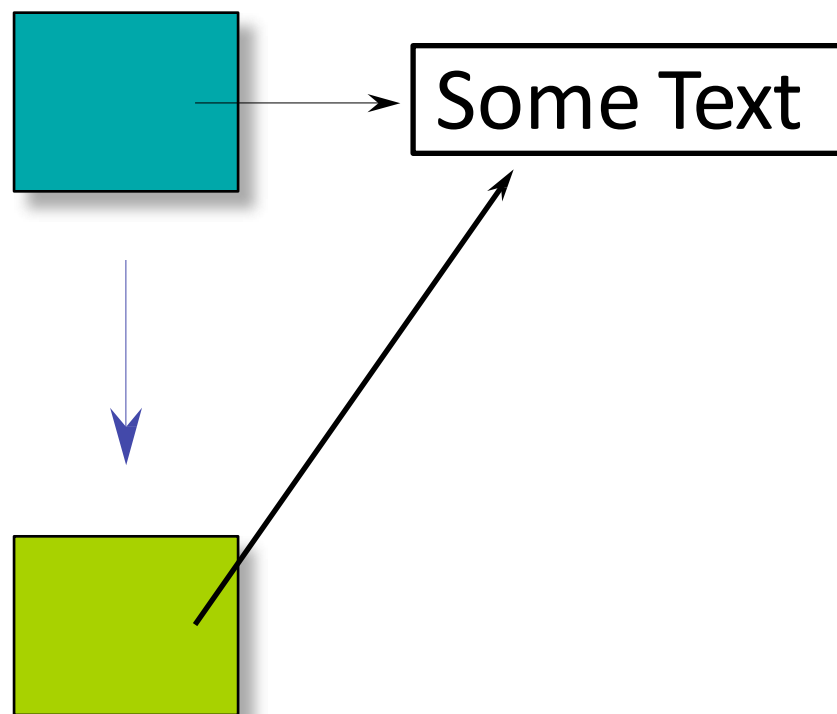
Copy pointer



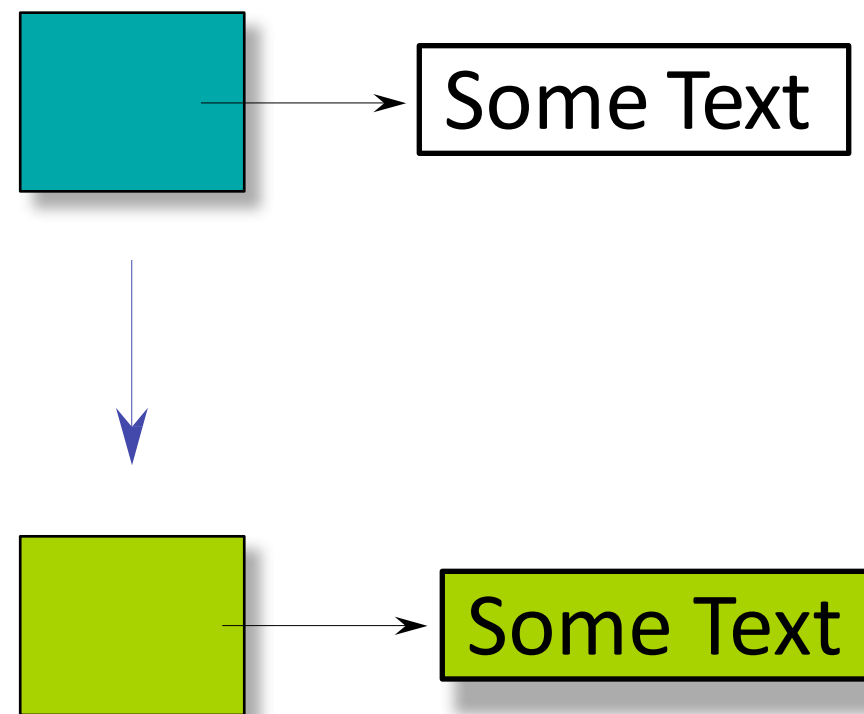
code & demo

Choices

Copy pointer

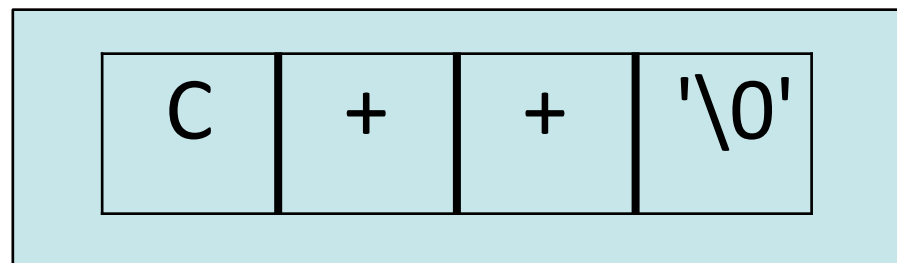


Copy entire block



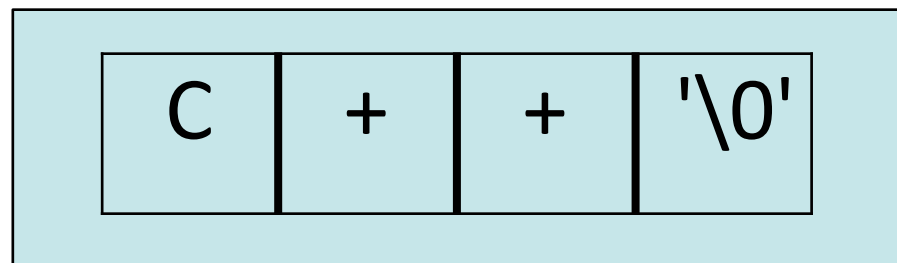
code & demo

Character strings



Character strings

- In C++, a character string is
 - An array of characters
 - With a special terminator — `'\0'` or ASCII null
- The string “C++” is represented, in memory, by an array of four (4, count'em) characters



Standard C library String fxns

- Declared in `<cstring>`

Standard C library String fxns

- Declared in `<cstring>`

```
size_t strlen(const char *s);
```

- s is a null-terminated string
- returns the length of s
- length does not include the terminator!

```
char *strcpy (char *dest, const char *src);
```

- Copies src to dest stopping after the terminating null-character is copied. (src should be null-terminated!)
- dest should have enough memory space allocated to contain src string.
- Return Value: returns dest

Person (char*) implementation

```
#include <cstring>           // #include <string.h>
using namespace std;

Person::Person( const char *s ) {
    name = new char[::strlen(s) + 1];
    ::strcpy(name, s);
}

Person::~~Person() {
    delete [] name;         // array delete
}
```

Person copy constructor

Person copy constructor

- To Person declaration add copy ctor prototype:

```
Person( const Person& w );    // copy ctor
```

- To Person .cpp add copy ctor definition:

```
Person::Person( const Person& w ) {  
    name = new char[::strlen(w.name) + 1];  
    ::strcpy(name, w.name);  
}
```


Person copy constructor

- To Person declaration add copy ctor prototype:

```
Person( const Person& w );    // copy ctor
```

- To Person .cpp add copy ctor definition:

```
Person::Person( const Person& w ) {  
    name = new char[::strlen(w.name) + 1];  
    ::strcpy(name, w.name);  
}
```

- No value returned
- Accesses w.name across client boundary
- The copy ctor initializes uninitialized memory

Person: string name

- What if the name was a string (and not a char*)

```
#include <string>
class Person {
public:
    Person( const string& );
    ~Person();
    void print();
    // ... other accessor fxns ...
private:
    string name;    // embedded object (composition)
    // ... other data members...
};
```

Person: string name...

Person: string name...

- In the default copy ctor, the compiler recursively calls the copy ctors for all member objects (and base classes).

Person: string name...

- In the default copy ctor, the compiler recursively calls the copy ctors for all member objects (and base classes).
- default is *memberwise* initialization

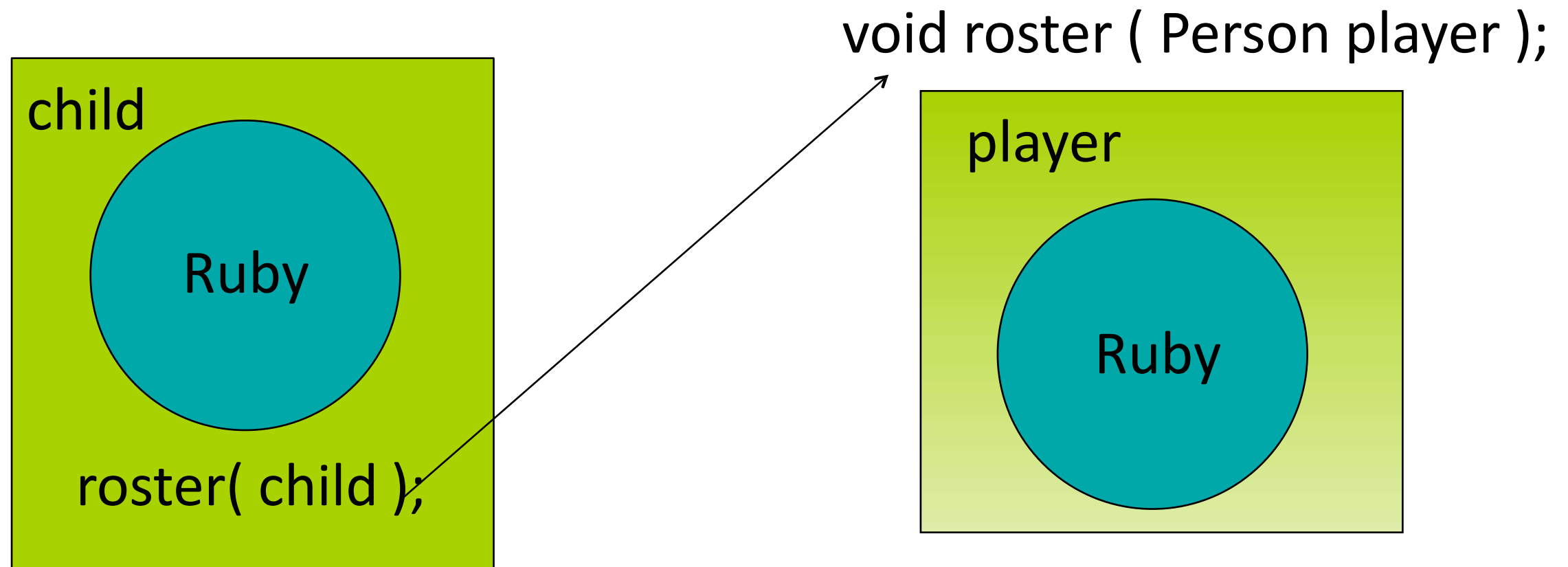
When are copy ctors called?

- During call by value

```
void roster( Person );           // declare function
```

```
Person child( "Ruby" );         // create object
```

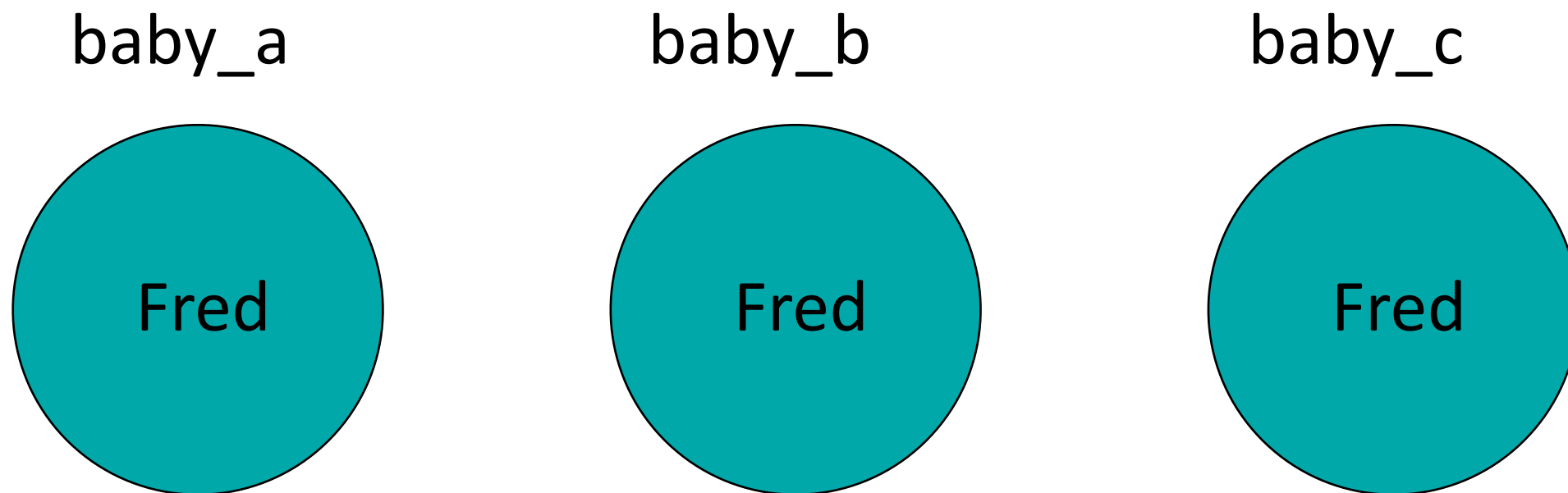
```
roster( child );                // call function
```



When are copy ctors called?

- During initialization

```
Person baby_a("Fred");  
// these use the copy ctor  
Person baby_b = baby_a;    // not an assignment  
Person baby_c( baby_a );   // not an assignment
```



When are copy ctors called?

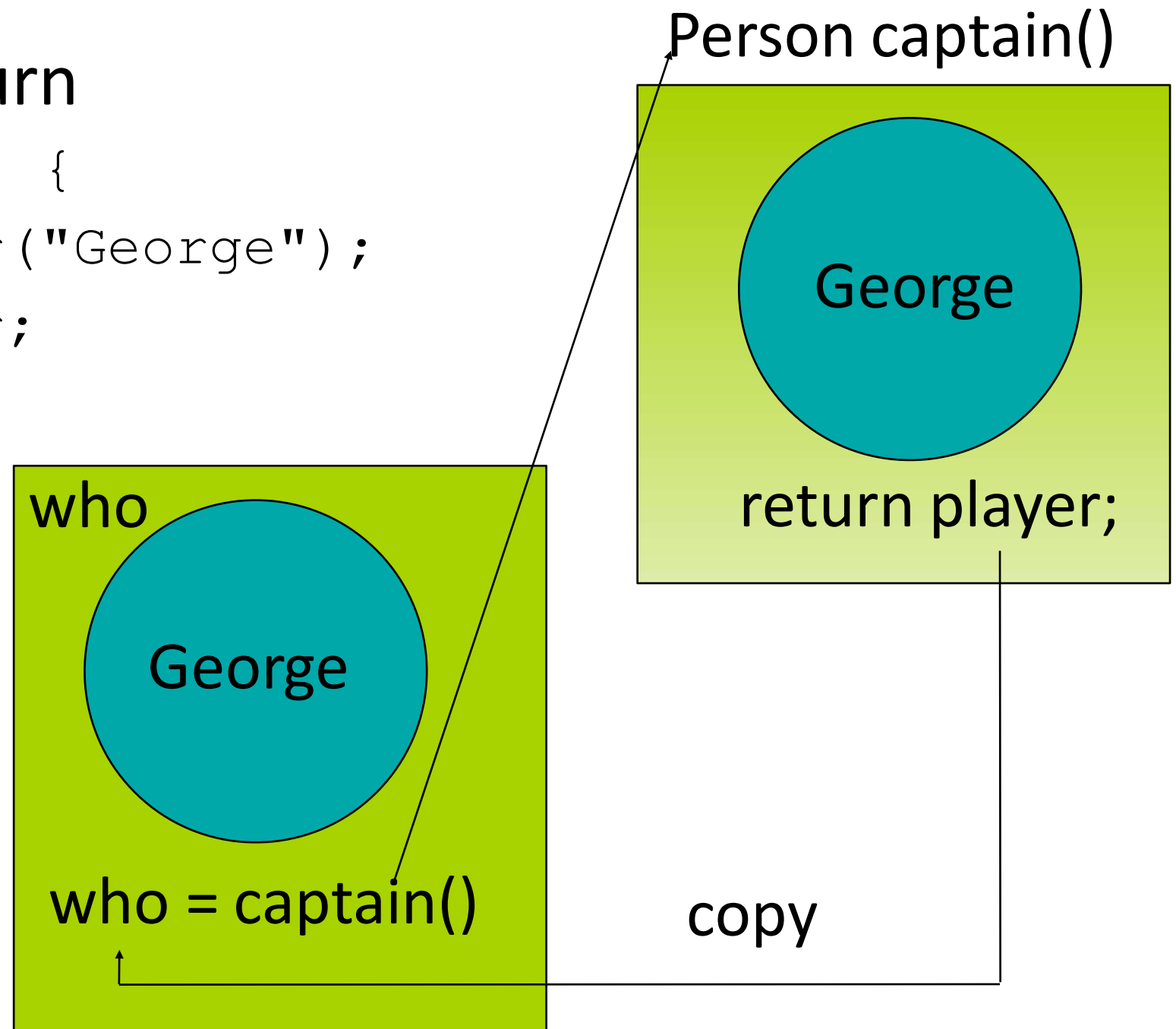
- During function return

```
Person captain() {  
    Person player("George");  
    return player;  
}
```

...

```
Person who  
    = captain();
```

...



Copies and overhead

Copies and overhead

- Compilers can "optimize out" copies when safe!

Copies and overhead

- Compilers can "optimize out" copies when safe!
- Programmers need to
 - Program for “dumb” compilers
 - Be ready to look for optimizations

Example

```
Person copy_func( Person p ) {  
    p.print();  
    return p; // copy ctor called!  
}
```

```
Person nocopy_func( char *who ) {  
    return Person( who );  
} // no copy needed!
```

code & demo

Constructions vs. assignment

- Every object is constructed once
- Every object should be destroyed once
 - Forget to invoke `delete`
 - Invoke `delete` more than once
- Once an object is constructed, it can be the target of many assignment operations

Copy ctor guidelines

Copy ctor guidelines

- In most cases, you don't have to write.
- Be explicit when necessary, e.g., managing raw pointers.
 - create your own copy ctor
- If you don't need one declare a private copy ctor (no need to define the body).
 - prevents creation of a default copy constructor
 - generates a compiler error for copy

Copy ctor guidelines

- In most cases, you don't have to write.
- Be explicit when necessary, e.g., managing raw pointers.
 - create your own copy ctor
- If you don't need one declare a private copy ctor (no need to define the body).
 - prevents creation of a default copy constructor
 - generates a compiler error for copy
 - use `Person(const Person &rhs) = delete;`
(since C++11)

static

Static in C++

Two basic meanings

- Static storage
 - allocated once at a fixed address
- Visibility of a name
 - internal linkage

Uses of “static” in C++

Static free functions	Internal linkage
Static global variables	Internal linkage
Static local variables	Persistent storage
Static member variables	Shared by all instances
Static member function	Shared by all instances, can only access static member variables

Global static hidden in file

.cpp file 1

```
int g_global; ←  
static int s_local;
```

```
void  
func() {  
    ...  
}
```

```
static  
void  
hidden() { ... }
```

.cpp file 2

```
extern int g_global;  
void func();
```

```
extern int s_local;  
int myfunc() {
```

```
    g_global += 2;  
    s_local *= g_global;  
    func();  
}
```

?

Static inside functions

Static inside functions

- Value is remembered for entire program

Static inside functions

- Value is remembered for entire program
- Initialization occurs only once

Static inside functions

- Value is remembered for entire program
- Initialization occurs only once
- Example:
 - count the number of times the function has been called

```
void f() {  
    static int num_calls = 0;  
    ...  
    num_calls++;  
}
```


Static applied to objects

- Suppose you have a class

```
class X {  
    X(int, int);  
    ~X();  
    ...  
};
```

- And a function with a static X object

```
void f() {  
    static X my_X(10, 20);  
    ...  
}
```


Static applied to objects...

Static applied to objects...

- Construction occurs when definition is encountered
 - Constructor called at-most once
 - The constructor arguments must be satisfied
- Destruction takes place on exit from *program*
 - Compiler assures LIFO order of destructors

Conditional construction

- Example: conditional construction

```
void f(int x) {  
    if (x > 10) {  
        static X my_X(x, x * 21);  
        ...  
    }  
}
```

- `my_X`

- is constructed once, if `f()` is ever called with `x > 10`
- retains its value
- destroyed only if constructed

Global objects

Global objects

- Consider

```
#include "X.h"
static X global_x(12, 34);
static X global_x2(8, 16);
```

- Constructors are called before main() is entered
 - Order controlled by appearance in file
 - In this case, `global_x` before `global_x2`
 - main() is "no longer" the *first* function being called
- Destructors called when
 - main() exits
 - exit() is called

Can we apply static to members?

Can we apply static to members?

- Static means
 - Hidden
 - Persistent
- Hidden: *A static member is a member*
 - Obeys usual access rules
- Persistent: *Independent of instances*

Can we apply static to members?

- Static means
 - Hidden
 - Persistent
- Hidden: *A static member is a member*
 - Obeys usual access rules
- Persistent: *Independent of instances*
- Static members are class-wide
 - variables or
 - functions

Static members

Static members

- Static member variables
 - Global to all class member functions
 - Initialized once, at file scope
 - provide a place for this variable and init it in .cpp
 - No 'static' in .cpp
- Example: StatMem.h, StatMem.cpp

Static members

- Static member functions
 - Have no implicit receiver ("this")
 - (why?)
 - Can access only static member variables
 - (or other globals)
 - Can't be dynamically overridden
- Example: StatFun.h, StatFun.cpp

To use static members

- <class name>::<static member>
- <object variable>.<static member>

Container & STL

Object-Oriented Programming with C++

A personal notebook

- It allows notes to be stored.
- It has no limit on the number of notes it can store.
- It will show individual notes.
- It will tell us how many notes it is currently storing.

Collection

- Collection objects are objects that can store an arbitrary number of other objects.

What is STL

- STL = Standard Template Library
- Part of the ISO Standard C++ Library
- Data Structures and algorithms for C++.



Alexander Stepanov

Алекса́ндр Алекса́ндрович Степа́нов

Book:

From Mathematics to Generic Programming

Why should I use STL?

- Reduce development time.
 - Data-structures already written and debugged.
- Code readability
 - Fit more meaningful stuff on one page.
- Robustness
 - STL data structures grow automatically.
- Portable code.
- Maintainable code
- Easy

C++ Standard Library

- Library includes:
 - A **pair** class (pairs of anything, int/int, int/char, etc)
 - Containers
 - **vector** (expandable array)
 - **deque** (expandable array, expands at both ends)
 - **list** (double-linked)
 - **set** and **map**
 - Basic Algorithms (**sort**, **search**, etc)
- All identifiers in library are in **std** namespace
using namespace std;

The three parts of STL

- Containers

- Sequence Containers (vector, deque, list, forward_list, array, string)
- Associative Containers (set, multiset, map, multimap)
- Unordered Associative Containers (unordered_set, unordered_multiset, unordered_map, unordered_multimap)

- Algorithms

- Iterators

The 'Top 3' data structures

- map
 - Any key type, any value type.
 - Sorted.
- vector
 - Like c array, but auto-extending.
- list
 - doubly-linked list

All sequential containers

- vector: variable array
- deque: dual-end queue
- list: double-linked-list
- forward_list: as it
- array: as “array”
- string: char array

Example using the vector class

- Use “namespace std” so that you can refer to vectors in C++ library
- Just declare a vector of ints (no need to worry about size)
- Add elements
- Have a pre-defined iterator for vector class, can use it to print out the items in vector

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int main( ) {
```

```
    vector<int> x;
```

```
    for (int a=0; a<1000; a++)
```

```
        x.push_back(a);
```

```
    vector<int>::iterator p;
```

```
    for (p=x.begin(); p<x.end(); p++)
```

```
        cout << *p << " ";
```

```
    return 0;
```

```
}
```

Generic classes

```
vector<string> notes;
```

- Have to specify two types: the type of the collection itself (here: vector) and the type of the elements that we plan to store in the collection (here: string)

vector

- It is able to increase its internal capacity as required: as more items are added, it simply makes enough room for them.
- It keeps its own private count of how many items it is currently storing. Its size method returns the number of objects currently stored in it. 
- It maintains the order of items you insert into it. You can later retrieve them in the same order.

Class Exercises

- Write a code to test **vector**. Put 5000 items in the vector, and then prints out every fifth element
 - Element 0, element 5, element 10, etc.



Basic vector operations

- Constructors

```
vector<Elem> c;  
vector<Elem> c1(c2);
```

- Simple methods

```
V.size( )           // num items  
V.empty( )          // empty?  
==, !=, <, >, <=, >=  
v1.swap(v2)         // swap
```

- Iterators

```
I.begin( )          // first position  
I.end( )             // last position
```

- Element access

```
V.at(index)  
V[index]  
V.front( )           // first item  
V.back( )            // last item
```

- Add/Remove/Find

```
V.push_back(e)  
V.pop_back( )  
V.insert(pos, e)  
V.erase(pos)  
V.clear( )  
V.find(first, last, item)
```

Two ways to use Vector

- Preallocate

```
vector<int> v(100);  
v[80] = 1; // okay  
v[200] = 1; // bad
```

- Grow tail

```
vector<int> v2;  
int i;  
while (cin >> i)  
    v2.push_back(i);
```

Pay attention to efficiency

- Estimate and preserve the memory
- Avoid extra copies



list

- Same basic concepts as vector
 - Constructors
 - Ability to compare lists (`==`, `!=`, `<`, `<=`, `>`, `>=`)
 - Ability to access front and back of list
 - `x.front()`, `x.back()`
 - Ability to assign items to a list, remove items
 - `x.push_back(item)`, `x.push_front(item)`
 - `x.pop_back()`, `x.pop_front()`
 - `x.remove(item)`

Sample list application

- Declare a list of strings
- Add elements
 - Some to the back
 - Some to the front
- Iterate through the list
 - Note the termination condition for our iterator
p != s.end()
 - Cannot use **p < s.end()** as with vectors, as the list elements may not be stored in order

```
#include <iostream>
using namespace std;
#include <list>
#include <string>

int main( ) {
    list<string> s;
    s.push_back("hello");
    s.push_back("world");
    s.push_front("tide");
    s.push_front("crimson");
    s.push_front("alabama");
    list<string>::iterator p;
    for (p=s.begin(); p!=s.end(); p++)
        cout << *p << " ";
    cout << endl;
}
```

Maintaining an ordered list

- Declare a list
- Read in five strings, add them in order
- Print out the ordered list

```
#include <iostream>
using namespace std;
#include <list>
#include <string>
int main( ) {
    list<string> s;
    string t;
    list<string>::iterator p;
    for (int a=0; a<5; a++) {
        cout << "enter a string : ";
        cin >> t;
        p = s.begin();
        while (p != s.end() && *p < t)
            p++;
        s.insert(p, t);
    }
    for (p=s.begin(); p!=s.end(); p++)
        cout << *p << " ";
    cout << endl;
}
```


Example of List

```
#include <iostream>
#include <iterator>
#include <list>
list<int> L;
for(int i=1; i<=5; ++i)
    L.push_back(i);

//delete second item.
L.erase( ++L.begin() );

copy( L.begin(), L.end(),
      ostream_iterator<int>(cout, ","));
// Prints: 1,3,4,5,
```


map

- Maps are collections that contain pairs of values.
- Pairs consist of a key and a value.
- Lookup works by supplying a key, and retrieving a value.
- An example: a telephone book.

Using map

- A map with strings as keys and values

:map<string, string>

"Charles Nguyen"	"(531) 9392 4587"
"Lisa Jones"	"(402) 4536 4674"
"William H. Smith"	"(998) 5488 0123"

Simple Example of Map

```
map<long, int>    root;  
root[4]           = 2;  
root[1000000]     = 1000;  
long l;  
cin >> l;  
if (root.count(l))  
    cout<<root[l];  
else  
    cout<<"Not perfect square";
```

Simple Example of Map

```
map<long, int>root;  
root[4] = 2;  
root[1000000] = 1000;  
long l;  
cin >> l;  
if (root.count(l))  
    cout<<root[l];  
else  
    cout<<"Not perfect square";
```

Iterators

- Declaring
list<int>::iterator li;
- Front of container
list<int> L;
li = L.begin();
- Past the end
li = L.end();

Iterators

- Can increment

```
list<int>::iterator li;
```

```
list<int> L;
```

```
li=L.begin();
```

```
++li; // Second thing;
```

- Can be dereferenced

```
*li = 10;
```

Algorithms

- Take iterators as arguments

```
list<int> L;
```

```
vector<int> V;
```

```
// put list in vector
```

```
copy(L.begin(),
```

```
      L.end(),
```

```
      V.begin());
```



Typedefs

- Annoying to type long names

- `map<Name, list<PhoneNum>> phonebook;`
- `map<Name, list<PhoneNum>>::iterator finger;`

- Simplify with typedef

- `typedef PB map<Name, list<PhoneNum>>;`
- `PB phonebook;`
- `PB::iterator finger;`

- Easy to change implementation.

Typedefs

- Annoying to type long names

- `map<Name, list<PhoneNum>> phonebook;`
- `map<Name, list<PhoneNum>>::iterator finger;`

- Simplify with typedef

- `typedef PB map<Name, list<PhoneNum>>;`
- `PB phonebook;`
- `PB::iterator finger;`

- Easy to change implementation.

- C++ 11: *auto, using*

Using your own classes

- Might need:
 - Assignment Operator, `operator=()`
 - Default Constructor
- For sorted types, like `set, map, ...`
 - Need less-than operator: `operator<()`
 - Some types have this by default:
 - `int, char, string`
 - Some do not:
 - `char *`

Example of user-defined type

- Sorted container needs sort function.

```
struct full_name {  
    char * first;  
    char * last;  
    bool operator<(full_name & a) {  
        return strcmp(first, a.first) < 0;  
    }  
}  
map<full_name,int> phonebook;
```

Pitfalls

- Accessing an invalid `vector<>` element.

```
vector<int> v;
```

```
v[100]=1; // Whoops!
```

Solutions:

- use `push_back()`
- Preallocate with constructor.
- Reallocate with `resize()`
- Check `size()`

Pitfalls

- Inadvertently inserting into `map<>`

```
if (foo["bob"]==1)
// silently created entry "bob"
```

Use `count()` to check for a key without creating a new entry.

```
if (foo.count("bob"))
```

Or `contains()` introduced in C++20

```
if (foo.contains("bob"))
```

Pitfalls

- Using `empty()` on `list<>`

- Slow

- ```
if (my_list.size() == 0) {...}
```

- Fast

- ```
if (my_list.empty())    {...}
```



Pitfalls

- Using invalid iterator

```
list<int> L; 
list<int>::iterator li;
li = L.begin();
L.erase(li);
++li; // WRONG
```

- Use return value of erase to advance

```
li = L.erase(li); // RIGHT
```


Other data structures

- set, multiset, multimap
- queue, priority_queue
- stack, deque
- slist, bitset, valarray

